

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

30 - Processor

Introduction

- Principles and techniques used in implementing a processor
- Build up a **datapath** and constructs a simple version of a processor sufficient to implement an instruction set like MIPS
- More realistic pipelined MIPS implementation

Basic MIPS Implementation

- Implementation includes subset of core MIPS instruction set
 - Memory-reference instructions *load word* (lw) and *store word* (sw)
 - Arithmetic-logical instructions add, sub, AND, OR, and slt
 - Instructions *branch equal* (beq) and *jump* (j)
- It illustrates key principles used in creating a **datapath** and designing the **control**
- Implementation of the remaining instructions is similar

Program Counter

- PC
- Register containing the address of the instruction in the program being executed
- 32-bits

Implementation Overview

- For every instruction, the first two steps are identical:
 1. Send *program counter* (PC) to memory that contains code and fetch the instruction from that memory
 2. Read one or two registers, using fields of instruction to select the registers to read
 - For *lw* instruction, need to read only one register
 - But most other instructions require reading two registers

Implementation Overview

- After these two steps, actions required to complete instruction depend on instruction class
- Fortunately, for each of the three instruction classes (memory-reference, arithmetic-logical, and branches), the actions are largely the same, independent of the exact instruction

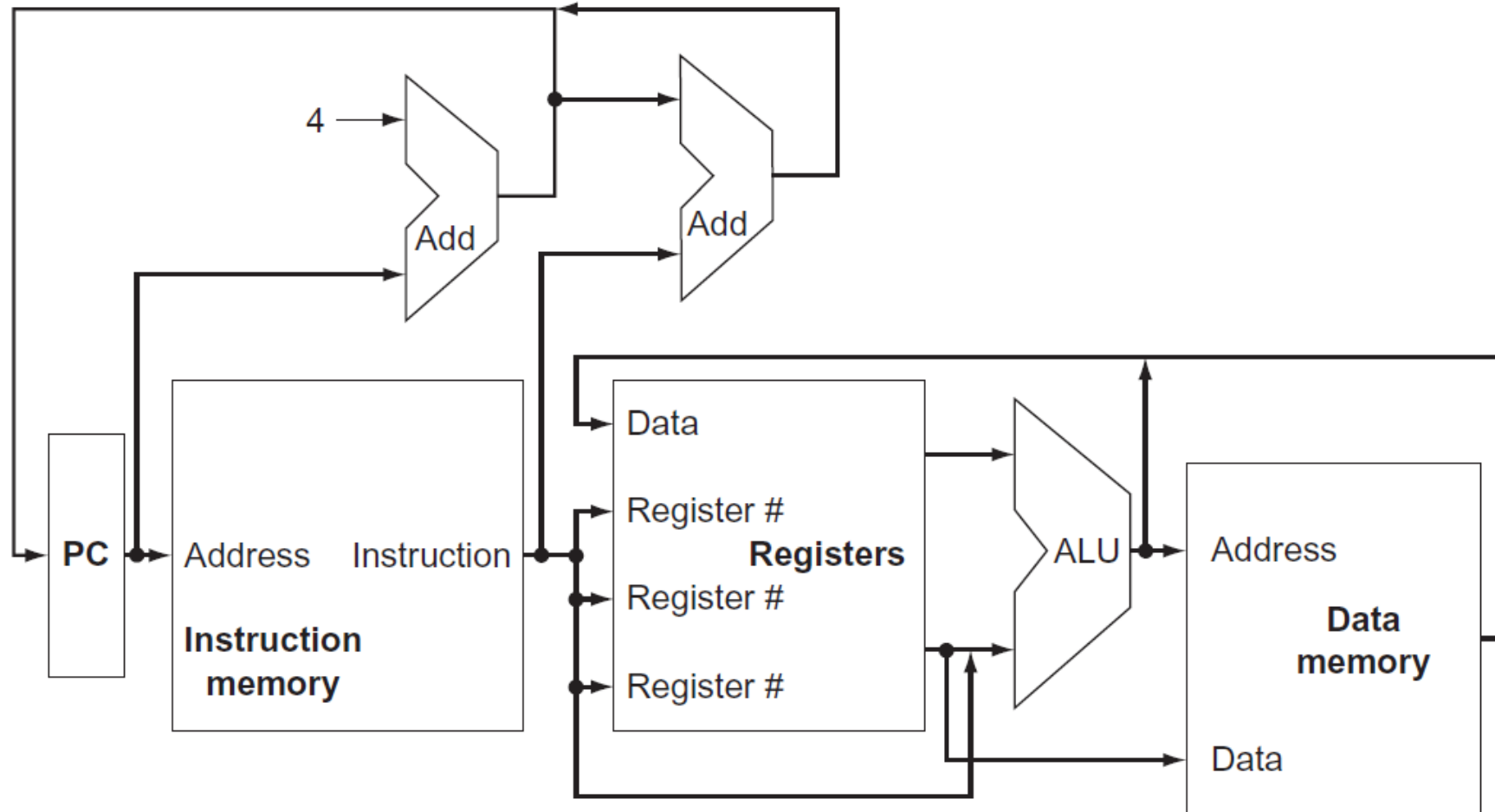
Implementation Overview

- Simplicity and regularity of MIPS IS simplifies implementation by making execution of many of instruction classes similar
- For example
 - All instruction classes, except jump, use ALU after reading registers
 - Memory-reference instructions use ALU for an address calculation
 - Arithmetic-logical instructions use ALU for operation execution
 - Branches use ALU for comparison

Implementation Overview

- After using ALU, actions required to complete various instruction classes differ
 - A memory-reference instruction will need to access memory either to read data for a load or write data for a store
 - Arithmetic-logical or load instruction must write data from ALU or memory back into a register
 - Branch instruction, may need to change next instruction address based on comparison
 - Otherwise, PC should be incremented by 4 to get address of next instruction

Implementation Overview



Multiplexors

- In several places, figure shows data going to a particular unit as coming from two different sources
- In practice, these data lines cannot simply be wired together
 - must add a logic element that chooses from among multiple sources and steers one of those sources to its destination
 - Selection is commonly done with a **multiplexor**
- Control lines are set based primarily on information taken from the instruction being executed

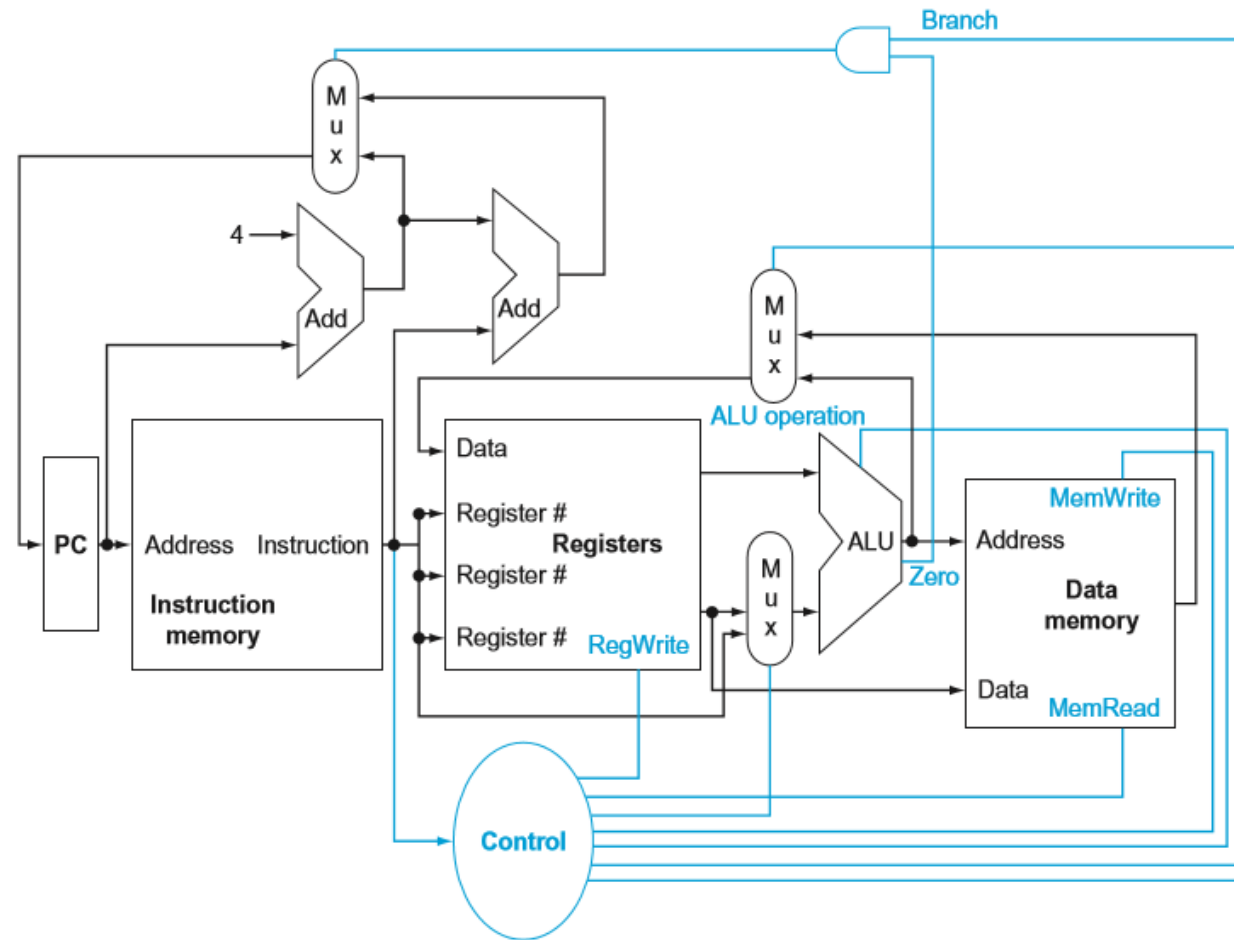
Control

- Several of the units must be controlled depending on the type of instruction
 - For example
 - Data memory must read on a load and written on a store
 - Register file must be written only on a load or an arithmetic-logical instruction
 - ALU must perform one of several operations
- Like the multiplexors, control lines that are set on the basis of various fields in the instruction direct these operations

Control Unit

- A control unit, which has instruction as an input, is used to determine how to set the control lines for the functional units and two of the multiplexors
- Third multiplexor, which determines whether $PC + 4$ or branch destination address is written into PC, is set based on the *Zero* output of ALU, which is used to perform the comparison of a *beq* instruction

Control



Logic Design Convention

- We will use the word
 - **asserted** to indicate a signal that is logically **high** and **assert** to specify that a signal should be driven logically **high**
 - **deassert** or **deasserted** to represent logically **low**
 - We use the terms **assert** and **deassert** because when we implement hardware, at times 1 represents logically high and at times it can represent logically low

Logic Design Convention

- An edge-triggered methodology allows us to read contents of a register, send the value through some combinational logic, and write that register in the same clock cycle
- We use **rising clock edge**